

Aujourd'hui

- Présentation des classes (fin)
 - Autres outils de programmation objet
 - Destructeur
- Etude d'une classe Tableau

Autres outils utiles en programmation objet

- Méthodes en ligne
- Passage d'objets en paramètre
- Sur-définition de méthodes
- Redéfinition d'opérateurs
- Méthodes "const"

Passage d'objets en paramètre

- Privilégier le passage par référence
- Evite une recopie d'objet à l'appel de la méthode
- Utiliser le mot clef *const* pour les paramètres d'entrée

```
class Fraction
{
    ...
public:
    ...
    Fraction ajouter(const Fraction &f);
};
```

Paramètres de retour

- Passage par valeur : génère une copie d'objet lors du retour dans la fonction appelante
- A éviter lorsque c'est possible (pour les objets volumineux)
- Peut être remplacé par un paramètre de sortie passé par référence

Paramètre de retour

```
// Fraction .h

class Fraction
{
    ...
public:
    ...
    void ajouter(const Fraction &f1, Fraction &f2);
};
```

```
// Fraction.cpp

void Fraction::ajouter(const Fraction &f1, Fraction &f2)
{
    f2.numerateur_ = numerateur_ * f1.denominateur_
                      + f1.numerateur_ * denominateur_;
    f2.denominateur_ = denominateur_ * f1.denominateur_;
}
```

Sur-définition de méthodes

- Définition de plusieurs méthodes ayant le même nom mais des paramètres différents
- La bonne méthode est choisie en fonction du nombre et du type des paramètres

Exemples de surdéfinitions

```
// Fichier fraction.h

class Fraction
{
    int numerateur_;
    int denominateur_;

public:
    Fraction(int n=0,int d=1);
    Fraction ajouter(const Fraction &f);
    Fraction ajouter(int val);
    Fraction ajouter(double val);
};
```

```
// main (suppose le fraction.cpp correctement écrit)

Fraction f1(1,2),f2(3,4);
f1=f1.ajouter(f2);      // Version 1
f1=f1.ajouter(10);      // Version 2
f1=f1.ajouter(3.14);    // Version 3
```

Redéfinition d'opérateurs

- Possibilité de définir des méthodes associées aux opérateurs du langage C++
- Sont appelées automatiquement à la place de l'opérateur correspondant

Redéfinition d'opérateurs

```
// Fraction .h

class Fraction
{
    ...
public:
    ...
    Fraction operator+(const Fraction &f);
};
```

```
// Fraction.cpp

Fraction Fraction::operator+(const Fraction &f)
{
    Fraction resultat;
    resultat.numerateur_ = numerateur_ * f.denominateur_
                             + f.numerateur_ * denominateur_;
    resultat.denominateur_ = denominateur_ * f.denominateur_;
    return resultat;
}
```

Utilisation de l'opérateur +

```
// Fichier main.cpp

#include <iostream>
#include "fraction.h"

using namespace std;

int main()
{
    Fraction f1(1,2), f2(3,4), f3;

    f3=f1+f2;    // <=> f3=f1.operator+(f2);

    cout<<"Résultat : ";
    f3.afficher();
    return 0;
}
```

Méthodes "const"

- Indiquer au compilateur que les champs de la classe ne seront pas modifiés par une méthode
- Cette méthode fait simplement une lecture des champs

Déclaration complète de la classe

```
// Fichier fraction.h

class Fraction
{
    int numerateur_;
    int denominateur_;

public:

    Fraction(int numerateur=0,int denominateur=1);
    int getNumerateur() const {return numerateur_;}
    int getDenominateur() const {return denominateur_;}
    void saisir();
    void afficher() const;
    Fraction operator+(const Fraction &f) const;
};
```

Définition des méthodes

```
// Fichier fraction.cpp
#include<iostream>
#include "fraction.h"

using namespace std;

Fraction::Fraction(int numérateur,int denominateur)
{
    numérateur_=numérateur;
    denominateur_=denominateur;
}

void Fraction::saisir()
{
    cin>>numérateur_>>denominateur_;
}

void Fraction::afficher() const
{
    cout<<numérateur_<<" / "<<denominateur_<<endl;
}
```

Définition des méthodes

```
Fraction Fraction::operator+(const Fraction &f)
{
    Fraction resultat;
    resultat.numerateur_ = numerateur_ * f.denominateur_
                             + f.numerateur_ * denominateur_;
    resultat.denominateur_ = denominateur_ * f.denominateur_;
    return resultat;
}
```

Utilisation finale

```
// Fichier main.cpp

#include <iostream>
#include "fraction.h"

using namespace std;

int main()
{
    Fraction f1;
    cout<<"Entrer une fraction f1 : ";
    f1.saisir();

    Fraction f2(1,2);

    f1=f1+f2;

    cout<<"Résultat : ";
    f1.afficher();
    return 0;
}
```

Destructeur

- Méthode "spéciale" appelée automatiquement lorsqu'un objet est détruit
- Porte le nom de la classe précédé de ~
- N'a pas de paramètres ni de valeur de retour

Exemple de destructeur

```
class Exemple
{
    char nom_[20];
public:
    Exemple(const char nom[20]);
    ~Exemple();
};

Exemple::Exemple(const char nom[20])
{
    strcpy(nom_, nom);
    cout<<"Construction de "<<nom_<<endl;
}

Exemple::~~Exemple()
{
    cout<<"Destruction de "<<nom_<<endl;
}
```

Exemple de destructeur

```
int main()
{
    Exemple e1("e1");

    char s[2][20]={"e2","e3"};
    for(int i=0;i<2;i++)
    {
        Exemple e(s[i]);
    }

    return 0;
}
```

// Exécution

```
Construction de e1
Construction de e2
Destruction de e2
Construction de e3
Destruction de e3
Destruction de e1
```

Aujourd'hui

- Présentation des classes (fin)
 - Autres outils de programmation objet
- Etude d'une classe Tableau

Objectifs pédagogiques

- Etude d'une classe `TableauInt`
 - Comprendre le mécanismes de gestion d'un tableau dynamique
 - Savoir manipuler les tableaux dynamiques
- Présentation du modèle *vector* (STL)
 - Savoir utiliser ce modèle de classe
 - Utilisé dans la suite du cours et dans les exercices

Pourquoi une classe tableau ?

- Encapsuler la gestion dynamique
- Déclarer des tableaux de taille quelconque
- S'adapter à tout type d'élément
- Pouvoir changer la taille

Fonctionnalités de TableauInt

- Déclarer un tableau
 - Appel au constructeur
- Connaître la taille
 - Méthode getTaille
- Accéder à un élément
 - Méthode getElement
- Affecter un élément
 - Méthode setElement

Utilisation envisagée

```
int main()
{
    int n,i;
    cout<<"Quelle taille voulez-vous ?";
    cin>>n;
    TableauInt t(n);
    for (i=0;i<t.getTaille();i++)
        t.setElement(i,2*i); // Initialisation du tableau
    cout<<"t :";
    for (i=0;i<t.getTaille();i++)
        cout<<t.setElement(i)<<" "; // Affichage des éléments
    cout<<endl;
    return 0 ;
}
```

Principe : gestion d'un tableau dynamique par la classe

```
int n=10;  
int *p;  
p=new int[n];  
p[0]=1;  
p[1]=2;  
cout<<p[0]<<endl;  
cout <<p[1]<<endl;  
delete [] p;
```

Champs

Constructeur

Méthode setElement

Méthode getElement

Destructeur

Encapsulation

- L'objet `TableauInt` gère
 - Le tableau dynamique
 - Le pointeur associé

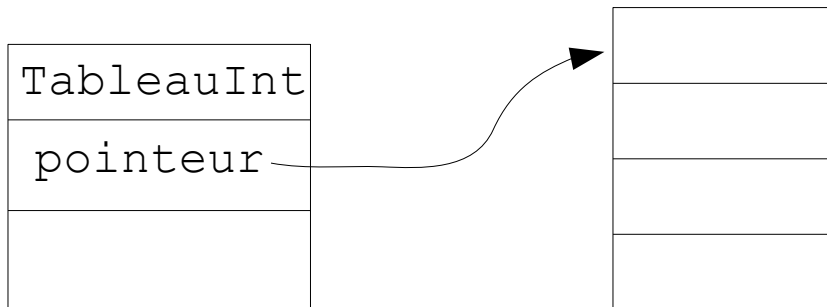


Tableau dynamique

Diagramme UML

TableauInt
tableau_ : int * taille_ : int
TableauInt(taille:int) ~TableauInt() getTaille() : int getElement(i:int) : int setElement(i:int, val:int)

Déclaration

```
class TableauInt
{
    int * tableau_;
    int  taille_;
public :
    TableauInt(int taille=0);
    ~TableauInt();
    int getTaille() const {return taille_;}
    int getElement(int i) const;
    void setElement(int i,int val);
};
```

Constructeur et destructeur

```
TableauInt::TableauInt(int taille)
{
    if (taille<0) taille=0;
    taille_=taille;
    tableau_=new int[taille_];
}
```

```
TableauInt::~~TableauInt()
{
    delete [] tableau_;
}
```

Méthodes d'accès aux éléments

```
int TableauInt::getElement(int i) const
{
    if (i<0 || i>=taille_)
    {
        cerr<<"Dépassement d'indice : i = "<<i;
        abort();
    }
    return tableau_[i];
}
```

```
void TableauInt::setElement(int i,int val)
{
    if (i<0 || i>=taille_)
    {
        cerr<<"Dépassement d'indice : i = "<<i;
        abort();
    }
    tableau_[i]=val;
}
```

Exemple d'utilisation

```
void main()
{
    int n,i;
    cout<<"Quelle taille voulez-vous ?";
    cin>>n;
    TableauInt t(n);
    for (i=0;i<t.getTaille();i++)
        t.setElement(i,2*i); // Initialisation du tableau
    cout<<"t :";
    for (i=0;i<t.getTaille();i++)
        cout<<t.getElement(i)<<" "; // Affichage des éléments
    cout<<endl;
}
```

Exercice 2

La classe PileInt permet la gestion d'une pile d'entiers. Elle encapsule un tableau dynamique dont la taille est fixée à la déclaration de la pile. Elle devra comporter :

- un constructeur et un destructeur,
- une méthode « empiler » pour ajouter une valeur au sommet de la pile (retourne une erreur si la pile est pleine),
- une méthode « depiler » pour enlever une valeur du sommet et la retourner,
- une « estVide » indiquant si la pile est vide.

a) Déclarer et définir la classe

b) Ecrire un programme illustrant l'utilisation des méthodes